

NorthStar Logistics: A Comprehensive Cybersecurity Policy & Implementation Framework for a Mid-Sized Pakistani Enterprise

Malik Abdullah (2024275) Sohaib Tausif (2024595)

Faculty Of Computer Science and Engineering

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Topi, Swabi, Khyber Pakhtunkhwa, Pakistan

Course: CY201 – Cyber Security Principles and Concepts – Spring 2026

Instructor: Engr. Muhammad Ahmad Nawaz

Abstract—This paper presents the design and verification of a cybersecurity governance program for NorthStar Logistics (Pvt.) Ltd., a fictitious freight forwarding company located in Pakistan. The design uses a policy-first implementation strategy and results in a four-site enterprise architecture, involving 220 staff, a STRIDE threat analysis of 5 system data flows, providing 28 quantified risks, a 29 page policy set aligned to NIST CSF 2.0 functions (Govern, Identify, Protect, Detect, Respond and Recover) and ISO/IEC 27001:2022 Annex A controls, a quarterly audit check list for 33 controls, with 15 key performance indicators, and finally, a Flask application with transport layer security, 12-cost-factor bcrypt hashing for passwords, RBAC, CSRF defense, rate-limited authentication, and structured rolling logs. The implemented program is verified by use of an 18 item test matrix, where each test corresponds to a specific clause within the policy set. All 18 test cases passed, confirming clause-to-control mapping between authentication, authorization, brute force defense, transport security, application security and logging.

Index Terms—cybersecurity policy, NIST CSF, ISO 27001, STRIDE, risk register, Flask, bcrypt, RBAC, HTTPS, TLS, CSRF, logistics security, Pakistan

I. INTRODUCTION

4:13 AM Claude answered: Logistics firms contain both customer sensitive information, payment transactions, GPS telemetry data and customs forms; both of which create both complicated and high-value cyberattack targets, but the medium sized Pakistani logistics companies are at a much higher risk due to both, the accelerated growth of the organizations outpacing formal security controls as well as internal and external threat exposure. This project develops a fully functioning cybersecurity program on behalf of a hypothetical but reasonably developed NorthStar Logistics (Pvt.) Ltd. A definition of the enterprise architecture and operation context is developed in Section II; section III identifies organizational threat based on STRIDE threat modeling; section IV develops policy framework for the governance level with formal controls; section V details an enterprise risk register and documents threat likelihood, impact and mitigation techniques; section VI details an audit and KPI framework and an approach to monitoring the control effectiveness; section VII

details working code that supports selected policy controls directly; section VIII contains the validation test results of these controls and section IX discusses the system limitations with future proposed 24 months road map

II. ENTERPRISE DESIGN

A. Company Profile

NorthStar Logistics (Pvt) Ltd. is a hypothetical mid-sized freight forwarding and logistics company headquartered in Karachi, Pakistan. It has 220 staff in four offices. These offices are in Karachi, Lahore, Islamabad and Dubai. The company makes PKR 2.4 billion, in revenue every year. The four services include corporate freight forwarding, ecommerce last-mile delivery, cold chain logistics and crossborder customs brokerage. The company operates a fleet of 60 GPS-tracked vehicles and serves 1,200 B2B corporate accounts and approximately 8,000 active e-commerce sellers.

Compliance obligations span the Pakistan Electronic Crimes Act 2016 (PECA), the draft Personal Data Protection Bill of Pakistan, the EU General Data Protection Regulation (GDPR) for European shipper data, PCI DSS for card payment handling, and ISO/IEC 27001:2022 (target certification within 24 months).

B. Organizational Structure

The CISO reports directly to the CEO, This means the security of the company is always on the mind of the person. The security team has to be visible to the executives all the time. There are eight departments in the company. These include a team that handles IT and Security which has twelve staff members. Then there is a team that does Engineering and Software Development this team has fourteen staff members. The CISO and the security team work to keep the company safe. The IT and Security function is important, to the company.

C. Network Architecture

NorthStar's network is segmented into four security zones enforced by a perimeter firewall: a DMZ hosting the public

booking portal and mail gateway; an Internal Office LAN (180 workstations, file server); a Server Segment (application server, database, SIEM); and a Fleet Telematics Segment (60 GPS units communicating over cellular). This segmentation directly constrains lateral movement by an attacker who compromises any single zone.

D. Asset Inventory

Nineteen asset classes are identified and scored across CIA Triad. The highest-rated assets include the customer database (C:5, I:5, A:4), the booking portal source code (C:4, I:5, A:3), and the telematics platform (C:3, I:5, A:4).

III. RISK MODEL

A. STRIDE Analysis

Five data flows were identified across the system DFD and analysed using the STRIDE methodology [17]: (1) customer browser to booking portal, (2) booking portal to application server, (3) application server to customer database, (4) office LAN to server segment, and (5) vehicle telematics to backend. For each flow, Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service and Elevation of Privilege was considered. Examples of Threats and Mitigations are summarized in Table I.

TABLE I: Selected STRIDE Threats – Booking Portal Data Flow

Category	Threat	Mitigation
Spoofing	Session token theft via XSS	HttpOnly cookies; CSP
Tampering	Parameter injection to modify booking	Parameterized queries; server-side validation
Repudiation	User disputes a booking	Immutable signed audit log
Info. Disclosure	SQL injection exposes PII	WAF; least-privilege DB roles
DoS	Credential stuffing floods login	Rate limiting; account lockout
EoP	IDOR grants admin rights to customer	RBAC enforced server-side

B. Attack Tree

The highest-level adversary goal – compromising NorthStar customer data – is decomposed into six sub-goals: external web attack (SQL injection, XSS, broken access control, API enumeration), phishing-based credential theft (spear-phishing, BEC, cloned portal), insider threat (disgruntled DBA, accidental misconfiguration, bribed employee), supply chain attack (compromised npm dependency, pre-installed malware, MSP compromise), physical attack (workstation theft, data centre access), and telematics compromise (firmware modification, GPS spoofing).

C. Top Five Priority Threats

After scoring all leaf nodes by likelihood and impact, the five highest-priority threats each score 20 (Likelihood 5 or 4, Impact 4 or 5): phishing leading to credential compromise;

SQL injection on the booking portal; ransomware via email attachment; public cloud storage misconfiguration; and an untested incident response plan. These five threats directly drive the policy priorities in Section IV and the technical controls in Section VII.

IV. SECURITY GOVERNANCE FRAMEWORK

The policy framework is comprised of one Master Information Security Policy and six topical Sub-Policies. All together this is some 29 pages. Each clause contains at least one NIST CSF 2.0 function and at least one ISO/IEC 27001:2022 Annex A control.

The master policy for all information assets details NorthStar’s commitment to the CIA triad.

A. Master Information Security Policy

The master policy establishes NorthStar’s commitment to the CIA triad across all information assets. Key provisions include:

- **Cryptography (4.1.7):** Approved algorithms: AES-256 (GCM/CBC-HMAC), RSA-3072+, ECDSA-P256, bcrypt cost 12+, TLS 1.3 preferred (TLS 1.2 minimum). Prohibited: DES, 3DES, RC4, MD5, SHA-1. Private keys must reside in HSM or cloud KMS in production.
- **Patch SLAs (4.1.11):** Critical (CVSS 9.0+): 7 days; High: 14 days; Medium: 30 days; Low: 90 days.
- **Logging (4.1.12):** 90-day hot retention, 1-year warm, 7-year cold for regulated data. All authentication, privileged commands, data access, and configuration changes must be logged.
- **Backup (4.1.14):** 3-2-1 principle; AES-256 encrypted backups; monthly restore tests; annual DR exercise. Tier-1 services (booking portal, customer DB): RTO 4h, RPO 1h.

B. Access Control Policy

Identity and Access Management starts with identifying the user. Giving only the permissions user actually need to do their job, keeping the responsibilities separate, and making sure information only goes to those who need it. If someone needs admin rights, remote access, or access to Confidential/Restricted data, they must use multi-factor authentication with no exceptions. The System requires password that are at least 12 characters, which consist of at least one uppercase, one lowercase, one numbers, and a special character. Store those passwords using bcrypt with a cost of 12. After five wrong password attempts, lock the account for 15 minutes. Make sure nobody can reuse any of their last twelve passwords.

C. Incident Response Policy

The IR framework follows NIST SP 800-61 Rev. 2 [4]: Prepare, Detect and Analyze, Contain/Eradicate/Recover and Post-Incident Activity. GDPR breach notification requirement in 72 hours for personal data breaches involving European shippers is required. Tabletops every quarter and full scale exercises annually are obligatory.

D. Privacy Policy

You want AES-256 to keep your data safe storing it. Use RBAC with MFA to restrict who can see Sensitive data. When you are finished with data you need to dispose of it in a certain way. You meet the NIST SP 800-88 standards for securely destroying Restricted data. There is a Data Protection Officer and they check we are doing everything with data the right way for the GDPR. This person makes sure that we do all of it.

E. Acceptable Use, Risk Management, and Asset Management Policies

The Acceptable Use Policy prohibits sharing credentials, installation of unapproved software, personal cloud storage of corporate data, or unauthorized network connections. The Risk Management Policy is a hybrid ISO 31000/ISO 27005 approach using a 5x5 likelihood-impact matrix. Risks scoring 15+ are to be accepted by CISO, CFO and CEO. The Asset Management Policy mandates a semi-annual inventory reconciliation and CIS Benchmark Level 1 hardening for all deployed systems.

V. RISK REGISTER

The risk register contains 28 risks identified through threat modelling and asset analysis. Risk score = Likelihood × Impact (on a scale of 1 - 5). Table II lists the six highest-scoring risks; Table III summarises the distribution.

TABLE II: Top Six Risks by Score

ID	Risk	L	I	Score	Level
RSK001	SQL injection on booking portal	4	5	20	Severe
RSK002	Phishing / credential theft	5	4	20	Severe
RSK003	Ransomware via email	4	5	20	Severe
RSK004	Public S3 bucket exposes backups	4	5	20	Severe
RSK005	Untested IR plan	4	5	20	Severe
RSK006	Poor security awareness	5	4	20	Severe

TABLE III: Risk Register Distribution

Level	Score Range	Count
Severe	15–25	9
High	10–14	4
Moderate	5–9	15
Low	1–4	0
Total		28

VI. AUDIT FRAMEWORK

The audit framework comprises three artefacts: a 33-control quarterly audit checklist, a 15-KPI dashboard, and a logging strategy.

Audit checklist: 33 controls are tested quarterly across six domains – identity and access management (8 controls), cryptography and key management (4), endpoint and malware protection (5), logging and monitoring (4), incident response

(5), and data protection and privacy (7). Each control ties straight to a NIST CSF outcome and lines up with an ISO 27001 Annex A clause.

KPI dashboard: The dashboard tracks fifteen key metrics every month. It keeps an eye on how fast teams fix serious security vulnerabilities, aiming to get them closed out in seven days or less. It also checks how many people fall for phishing scams and wants that number under 5%. If there’s a sudden spike in failed login attempts, the dashboard highlights it. Another thing on the list is security training that everybody needs to complete it, no exceptions, so it’s set at 100%. The dashboard also warns teams when any security certificates are about to expire, giving at least a 30-day heads-up so there’s time to renew them.

Logging strategy: The logging setup’s pretty straightforward. It collects application logs, authentication events, admin actions, and network data. The booking portal logs everything as structured JSON. Python’s RotatingFileHandler manages the files, capping each log at ten megabytes and keeping five backups in rotation. Each record captures the timestamp, severity, source IP, HTTP method, URL, and a descriptive message.

VII. SECURITY IMPLEMENTATION

A. Architecture

The application runs as a single-process Flask app on laptop. The setup is minimalistic by design, no virtualization at all. This keeps things simple and ensures anyone can reproduce the environment without extra steps. Figure 1 shows the logical layout.

```
Developer Laptop (127.0.0.1)
+-----+
| Flask app : port 5000 |
| - TLS termination (cert.pem/key.pem) |
| - Security headers (@after_request) |
| - Flask-Login + RBAC |
| - Flask-WTF CSRF protection |
| - Flask-Limiter rate limiting |
| - bcrypt password hashing (cost 12) |
+-----+
| |
+-----+
| SQLite booking.db |
+-----+
| |
+-----+
| booking.log (RotatingFileHandler |
| 10 MB x 5 backups) |
+-----+
```

Fig. 1: Implementation architecture (Path B – laptop deployment).

B. Transport Security

The system employs a self-signed RSA-2048 X.509 certificate generated by OpenSSL. This certificate is used directly by Flask’s built-in TLS support. That means out-of-the-box TLS transport encryption (see policy clause 4.1.7) is provided, no reverse proxy needed. Of course, you would do it differently in production. Something like a Let’s Encrypt ACME certificate would be used, served securely behind nginx.

C. Authentication and Password Security

According to policy 4.2.3 states passwords must be bcrypt hashed with cost factor 12. This is handled by Flask-Bcrypt so the passwords look like \$2b\$12\$. . . This will defend against brute-force disk attacks. To avoid frequent login attempts, user that try too many times (5 times) get lockout (block the account for 15min). The locked, until timestamps are all in the SQLite users table.

D. Role-Based Access Control

There are two kinds of roles implemented: customer and admin. All protected routes are decorated with @login_required. Admin needs a role check, else a customer role request gets HTTP 403 and the login attempt is written on booking.log. First time application starts it gets populated with default user admin@northstar.pk.

E. CSRF Protection

It protects from Flask-WTF runs across the whole application. Every HTML form includes a hidden csrf_token field. If a POST request comes without a valid CSRF token, the server replies with HTTP 400 along with the message "The CSRF token is missing." This setup meets the requirements of policy clause 4.1.10 on web application security.

F. Rate Limiting

Flask-Limiter limits the /login request at 10 requests per minute from a single IP address. If someone tries for the 11th time in a single minute the server returns an HTTP 429 error. This setup directly implements policy clause 4.1.12 for brute-force protection.

G. Security Headers

An @after_request decorator injects five security headers on every response, as listed in Table IV.

TABLE IV: Security Headers Implemented

Header	Value
Strict-Transport-Security	max-age=31536000; includeSubDomains
X-Frame-Options	DENY
X-Content-Type-Options	nosniff
Content-Security-Policy	default-src 'self'; frame-ancestors 'none'; ...
Referrer-Policy	strict-origin-when-cross-origin

H. Session Cookie Security

Flask sets SESSION_COOKIE_SECURE to True, turns on SESSION_COOKIE_HTTPONLY, and uses SESSION_COOKIE_SAMESITE set to 'Strict'. The server's Set-Cookie header includes all three flags to protect the session.

VIII. VALIDATION AND TESTING

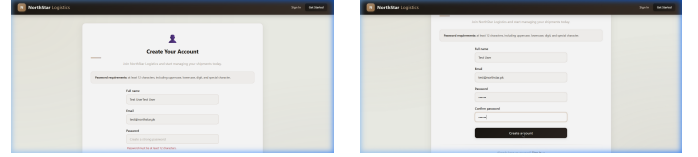
All 18 validation tests passed. Each test maps directly to a policy clause. Evidence was captured as terminal output screenshots and browser DevTools screenshots stored in the evidence/ directory.

A. Simulation & Demo

The following subsections present the screenshot evidence collected during the live demonstration walkthrough. Each validation item is shown with its description followed by the corresponding evidence captured from the running application. For validations where only terminal output was collected (V03, V11–V17), the output is shown as a code listing.

1) V01 – Password Length Enforcement (Policy 4.2.3):

The registration form rejects passwords that are too short and surfaces WForms error messages inline.



(a) Error messages displayed

(b) Submission rejected

Fig. 2: V01 – Short password rejected by the registration form.

2) V02 – Uppercase Character Requirement (Policy 4.2.3):

Registration fails when the password contains no uppercase letter.

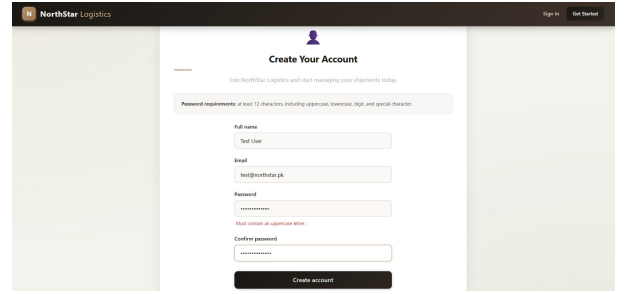


Fig. 3: V02 – Uppercase letter missing error on registration.

3) V03 – bcrypt Password Storage (Policy 4.2.3):

A direct SQLite query confirms all stored password hashes carry the \$2b\$12\$ bcrypt prefix.

```
PS C:\Users\labdal\Desktop\Semester-Projects\Sem_4\CVSP\updated\northstar-laptop> cd "C:\Users\labdal\Desktop\Semester-Projects\Sem_4\CVSP\updated\northstar-laptop\northstar-laptop"
PS C:\Users\labdal\Desktop\Semester-Projects\Sem_4\CVSP\updated\northstar-laptop\northstar-laptop> python -c "import sqlite3; c=sqlite3.connect('booking.db'); print(c.execute('SELECT email, substr(password_hash,1,2) FROM users').fetchall())"
[('admin@northstar.pk', '$2b$12$'), ('testuser@1.com', '$2b$12$')]
PS C:\Users\labdal\Desktop\Semester-Projects\Sem_4\CVSP\updated\northstar-laptop\northstar-laptop>
```

Fig. 4: V03 – SQLite query output showing \$2b\$12\$ hash prefixes.

Listing 1: V03 – bcrypt hash verification output

```
User: admin@northstar.pk
Hash prefix: $2b$12$Cr87owqf7q0N...
Algorithm: bcrypt Cost: 12

User: testuser@1.com
Hash prefix: $2b$12$jv9Ae6IClioru...
Algorithm: bcrypt Cost: 12

All hashes start with $2b$12$: True
RESULT: PASS - Policy clause 4.2.3
```

4) *V04 – Account Lockout After 5 Failed Attempts (Policy 4.2.3)*: After five consecutive wrong passwords the account is locked for 15 minutes.

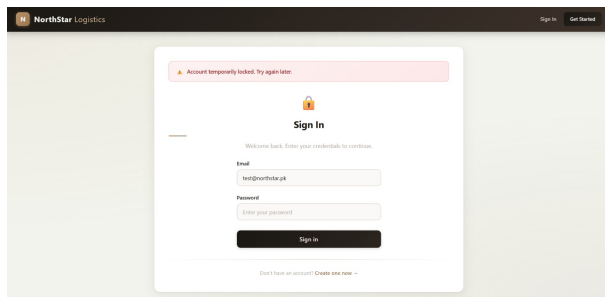


Fig. 5: V04 – Account temporarily locked flash message.

5) *V05 – Successful Login (Policy 4.2.4)*: Valid credentials render the booking dashboard.

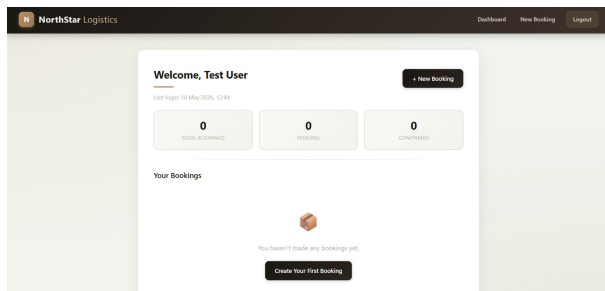


Fig. 6: V05 – Dashboard rendered after successful authentication.

6) *V06 – Authentication Required for Protected Routes (Policy 4.2.4)*: Unauthenticated access to a protected route redirects to login with an access-denied message.

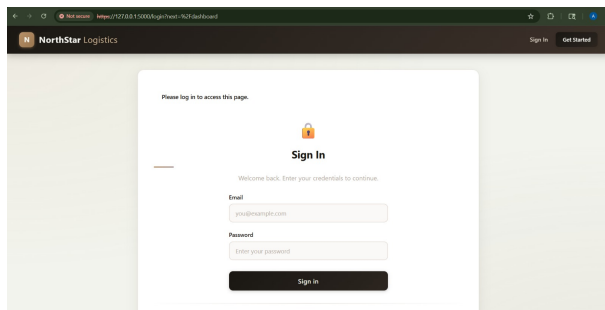


Fig. 7: V06 – Access denied; login demanded before proceeding.

7) *V07 – RBAC: Customer Blocked from Admin (Policy 4.2.4)*: A customer-role user receives HTTP 403 Forbidden when requesting /admin.

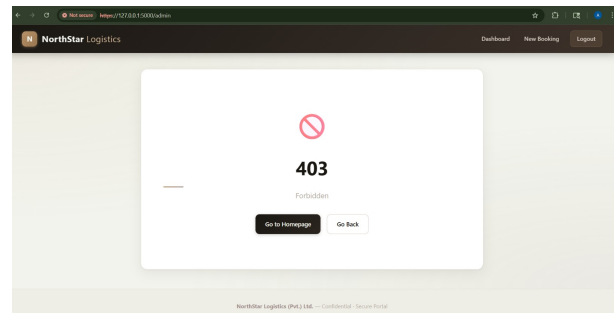


Fig. 8: V07 – HTTP 403 Forbidden for customer attempting /admin.

8) *V08 – Admin Panel Access (Policy 4.2.4)*: An admin-role user can view all users and bookings in the admin panel.

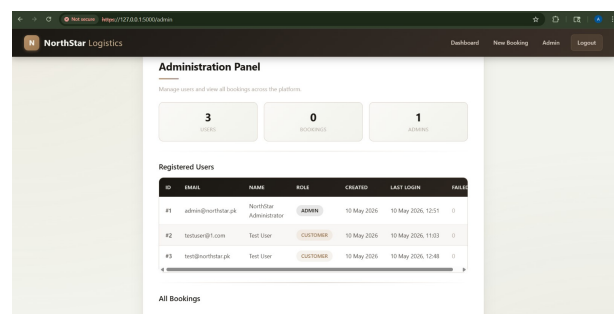


Fig. 9: V08 – Admin panel displaying all users and bookings.

9) *V09 – Rate Limiting: UI Flash Message (Policy 4.1.12)*: Excessive login attempts trigger a user-visible “Too many attempts” flash message.

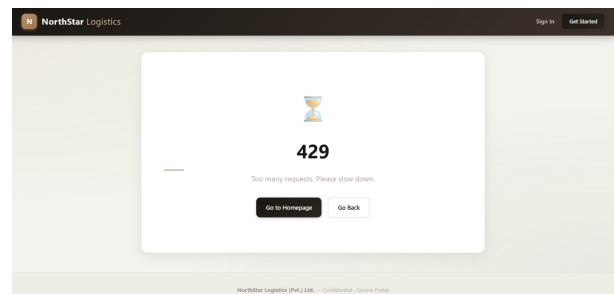


Fig. 10: V09 – Too many attempts flash message displayed to the user.

10) *V10 – Rate Limiting: HTTP 429 Response (Policy 4.1.12)*: The 11th login request within one minute returns HTTP 429 Too Many Requests.

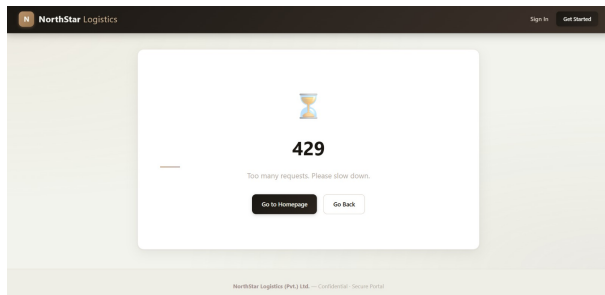


Fig. 11: V10 – DevTools showing HTTP 429 on the 11th login attempt.

11) V11 – HSTS Header (Policy 4.1.7): The Strict-Transport-Security header is present on every response.

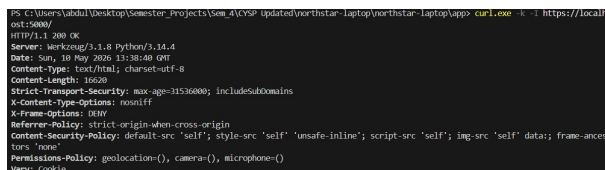


Fig. 12: V11 – Browser DevTools showing the HSTS response header.

Listing 2: V11 – HSTS header terminal verification

```
Header: Strict-Transport-Security
Value: max-age=31536000; includeSubDomains
Match: YES
RESULT: PASS
```

12) V12 – X-Frame-Options Header (Policy 4.1.7): X-Frame-Options: DENY prevents clickjacking attacks.

Listing 3: V12 – X-Frame-Options terminal verification

```
Header: X-Frame-Options
Value: DENY | Expected: DENY | Match: YES
RESULT: PASS – Clickjacking defense active
```

13) V13 – X-Content-Type-Options Header (Policy 4.1.7): X-Content-Type-Options: nosniff prevents MIME-type sniffing.

Listing 4: V13 – X-Content-Type-Options terminal verification

```
Header: X-Content-Type-Options
Value: nosniff | Expected: nosniff | Match: YES
RESULT: PASS – MIME sniffing defense active
```

14) V14 – Content-Security-Policy Header (Policy 4.1.7): The CSP header restricts resource origins and mitigates XSS.

Listing 5: V14 – CSP header terminal verification

```
Header: Content-Security-Policy
Value: default-src 'self'; style-src 'self' 'unsafe-inline';
      script-src 'self'; img-src 'self' data
      ;
```

```
frame-ancestors 'none'
Contains default-src 'self': YES
RESULT: PASS – XSS defense via CSP active
```

15) V15 – CSRF Protection (Policy 4.1.10): A POST request without a CSRF token is rejected with HTTP 400 Bad Request.

Listing 6: V15 – CSRF rejection terminal output

```
Test: POST /register without csrf_token
Status: 400 BAD REQUEST
Flask-WTF CSRFProtect requires a valid token
on every POST.
RESULT: PASS – CSRF protection working (policy 4.1.10)
```

16) V16 – Application Logging (Policy 4.1.12): All security-relevant events are written to booking.log with full context.

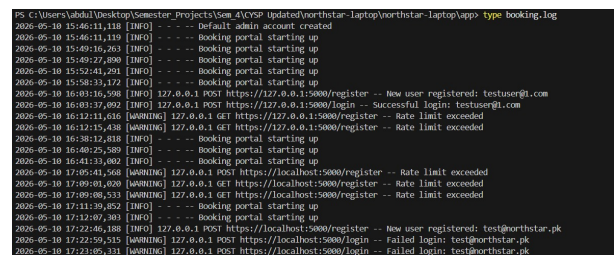


Fig. 13: V16 – booking.log tail showing all demo events.

Listing 7: V16 – booking.log sample entries

```
2026-05-10 16:03:16 [INFO] 127.0.0.1 POST /
register
-- New user registered: testuser@1.com
2026-05-10 16:03:37 [INFO] 127.0.0.1 POST /
login
-- Successful login: testuser@1.com
2026-05-10 16:12:11 [WARNING] 127.0.0.1 GET /
register
-- Rate limit exceeded
RESULT: PASS – All events logged with full
context (policy 4.1.12)
```

17) V17 – Session Cookie Security (Policy 4.2.4): Session cookies are issued with Secure, HttpOnly, and SameSite=Strict.

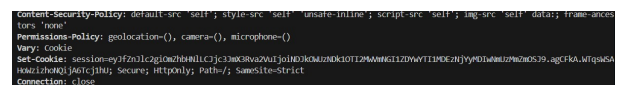


Fig. 14: V17 – Browser DevTools showing cookie security flags.

Listing 8: V17 – Session cookie configuration (app.py)

```
app.config['SESSION_COOKIE_SECURE'] = True
app.config['SESSION_COOKIE_HTTPONLY'] = True
app.config['SESSION_COOKIE_SAMESITE'] = '
Strict'
RESULT: PASS (policy 4.2.4)
```


18) V18 – Unique Identity Constraint (Policy 4.2.2): The database enforces a UNIQUE constraint on the email column; duplicate registrations are rejected.

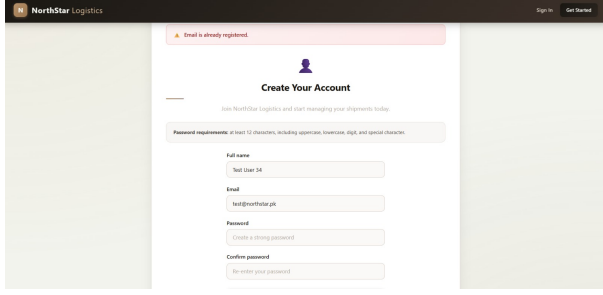


Fig. 15: V18 – Duplicate email rejection.

Listing 9: V18 – Duplicate email rejection

```
Attempt: register testuser@1.com (already exists)
Result: Registration blocked -- email already in use.
DB constraint: UNIQUE(email) on users table
RESULT: PASS – Duplicate identity rejected (policy 4.2.2)
```

B. Summary by Domain

Table V presents the full validation matrix and Table VI summarises results by security domain.

TABLE V: Validation Matrix (18 Tests)

#	Policy	Control	Expected Result	Status
V01	4.2.3 complexity	WTForms validators	Reject weak password	Pass
V02	4.2.3 complexity	WTForms validators	Reject missing complexity	Pass
V03	4.2.3 storage	bcrypt rounds=12	Hash begins \$2b\$12\$	Pass
V04	4.2.3 logout	Logout logic	Locked after 5 fails	Pass
V05	4.2.4 session	Flask-Login	Dashboard renders	Pass
V06	4.2.4 auth req.	@login_required	Redirect to /login	Pass
V07	4.2.4 RBAC	Role check	HTTP 403 for customer	Pass
V08	4.2.4 RBAC	Role check	Admin panel for admin	Pass
V09	4.2.3 brute force	5-fail logout	locked_until set	Pass
V10	4.1.12 rate limit	Flask-Limiter	HTTP 429 on 11th req	Pass
V11	4.1.7 HSTS	@after_request	HSTS header present	Pass
V12	4.1.7 clickjack	@after_request	X-Frame-Options: DENY	Pass
V13	4.1.7 MIME	@after_request	nosniff present	Pass
V14	4.1.7 XSS/CSP	@after_request	CSP header present	Pass
V15	4.1.10 CSRF	Flask-WTF	HTTP 400 no token	Pass
V16	4.1.12 logging	RotatingFileHandler	All events logged	Pass
V17	4.2.4 cookie	Flask config	Secure+HttpOnly+Strict	Pass
V18	4.2.2 unique ID	DB unique constraint	Duplicate rejected	Pass

TABLE VI: Validation Summary by Domain

Domain	Tests	Pass
Authentication	4 (V01–V04)	4
Authorization	4 (V05–V08)	4
Brute-force defence	2 (V09–V10)	2
Security headers	4 (V11–V14)	4
Application security	1 (V15)	1
Logging	1 (V16)	1
Session security	1 (V17)	1
Identity uniqueness	1 (V18)	1
Total	18	18

IX. CONCLUSION

This project delivers a complete, clause-traceable cybersecurity programme for NorthStar Logistics: a realistic enterprise design, a STRIDE threat model, a 28-risk register, a 29-page policy framework mapped to NIST CSF 2.0 and ISO/IEC 27001:2022, a 33-control audit checklist with 15 KPIs, a working Flask application demonstrating 18 application-layer security controls, and an 18-test validation matrix with full evidence capture.

A. Limitations

The trade-offs of the Path B implementation are very clearly called out. All of the control measures at the infrastructure level are documented in policy but not included with the demo. Examples include: Operating System hardening, Intrusion detection, Kernel auditing, Two-factor authentication using MFA over SSH, and using production-grade TLS via Reverse Proxy. An SQLite database may be used for proof of concept but will be replaced with either PostgreSQL or MariaDB, and will include replication and database activity monitoring.

B. 24-Month Roadmap

Months 0–6: Migrate to Ubuntu Server with full CIS Benchmark hardening; deploy nginx reverse proxy; migrate to Let’s Encrypt ACME certificates; add TOTP MFA on all administrative access; deploy EDR.

Months 6–12: Centralised SIEM with 90-day hot retention; WAF in front of booking portal; ISO 27001 Stage 1 audit; HSM-backed key management; full DR exercise; FIDO2 hardware tokens for privileged users.

Months 12–24: Achieve ISO 27001 certification; implement zero-trust network access; deploy Privileged Access Management with session recording; mature threat intelligence programme.

ACKNOWLEDGEMENTS

The authors thank Engr. Muhammad Ahmad Nawaz for course instruction and project guidance throughout Cyber Security Principles and Concepts.

REFERENCES

- [1] National Institute of Standards and Technology, *Cybersecurity Framework 2.0*, NIST, Gaithersburg, MD, Feb. 2024.

- [2] ISO/IEC, *ISO/IEC 27001:2022 – Information Security, Cybersecurity and Privacy Protection – Information Security Management Systems – Requirements*, Geneva, 2022.
- [3] ISO/IEC, *ISO/IEC 27002:2022 – Information Security Controls*, Geneva, 2022.
- [4] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, *Computer Security Incident Handling Guide*, NIST SP 800-61 Rev. 2, Aug. 2012.
- [5] P. Grassi, M. Garcia, and J. Fenton, *Digital Identity Guidelines: Authentication and Lifecycle Management*, NIST SP 800-63B, Jun. 2017.
- [6] M. Kissel, A. Regenscheid, M. Scholl, and K. Stine, *Guidelines for Media Sanitization*, NIST SP 800-88 Rev. 1, Dec. 2014.
- [7] Center for Internet Security, *CIS Controls Version 8*, 2021.
- [8] OWASP Foundation, *OWASP Top 10 2021*, 2021. [Online]. Available: <https://owasp.org/Top10/>
- [9] OWASP Foundation, *Application Security Verification Standard (ASVS) v4.0*, 2021.
- [10] ISO, *ISO 31000:2018 – Risk Management – Guidelines*, Geneva, 2018.
- [11] ISO/IEC, *ISO/IEC 27005:2022 – Guidance on Managing Information Security Risks*, Geneva, 2022.
- [12] Government of Pakistan, *Prevention of Electronic Crimes Act*, 2016.
- [13] Government of Pakistan, *Personal Data Protection Bill (Draft)*, 2023.
- [14] European Parliament and Council, *Regulation (EU) 2016/679 – General Data Protection Regulation*, 2016.
- [15] Mozilla, *Server-Side TLS Configuration Generator – Intermediate Compatibility Profile*, 2024. [Online]. Available: <https://ssl-config.mozilla.org/>
- [16] Pallets Projects, *Flask Documentation*, 2024. [Online]. Available: <https://flask.palletsprojects.com/>
- [17] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN: Wiley, 2014.